Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:
Студент 4 курса
Группы ИИ-26
Суховило В. С.
Проверила:
Хацкевич А. С.

Брест 2026

Цель работы

Практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQL Injection, XSS, BOLA/IDOR), исследования слабых JWT-токенов, а также проектирование и проверка механизмов защиты по рекомендациям OWASP (параметризованные SQL-запросы, проверка прав, CSP, защищённые Cookie).

Вариант №4

Исходные данные и индивидуальный вариант

Выполнение основано на учебном приложении labs/lab1/app.py из репозитория SMZKS-2026-autumn. Вариант определён по формуле (18 mod 15) + 1 = 4. Проверочный проект содержит уязвимую и исправленную версии приложения.

Параметр	**Значение**
SQL Injection	Error-based: инициирование диагностической ошибки SQLite
XSS	HTML-тег <body> с обработчиком onload
BOLA/IDOR	Профиль пользователя с ID = 5
JWT	Целевая роль administrator
CSP	Директива script-src с strict-dynamic

Среда и статус проверки

Исходный стенд написан на Flask 3.0 и PyJWT 2.8 (Python 3.10+). В предоставленном рабочем окружении библиотека Flask отсутствовала и её установка была недоступна по сети. Поэтому подтверждены реальные вычисления SQLite, PyJWT и Jinja2 и реальное исполнение HTML в Chromium; HTTP-ответы исходного Flask-приложения здесь не снимались. Для воспроизводимой проверки Flask создан автоматический тест tests/http_live_checks.py. Отчёт честно разделяет выполненные офлайн-проверки и ожидаемые по программной логике HTTP-результаты.

Задание 1. SQL Injection: обход аутентификации и устранение

Индивидуальное условие — Error-based SQLi. В исходной версии строка SQL собирается непосредственно из переменных username и password, после чего сообщение исключения SQLite возвращается клиенту. Это демонстрирует как возможность внедрения управляющего SQL, так и раскрытие диагностической информации.

```
query = f"SELECT id, username, role FROM users WHERE username='{username}'
```

```
AND password='{password}'"
c.execute(query)
```

Для провокации ошибки в учебном приложении использован ввод в поле username:

```
admin' AND nonexistent_sql_function()--
```

Во время фактической проверки SQLite получена ошибка «no such function: nonexistent_sql_function». В логике уязвимого Flask-маршрута данная ошибка возвращается со статусом HTTP 500. Чтобы получить токен для задания 4, дополнительно используется Boolean-based запрос — он не подменяет индивидуальный тип Error-based, а служит вспомогательным шагом.

```
' OR 1=1--
```

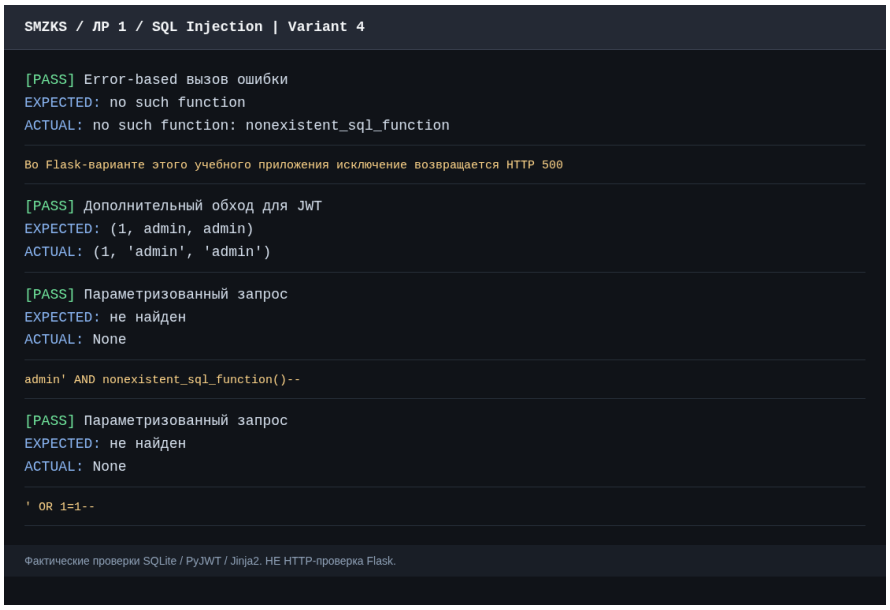


Рисунок 1 — Реальные результаты SQLite: ошибка БД, вспомогательный обход и проверка параметризации

Устранение: пользовательские данные передаются в sqlite3 через параметры-заполнители. Оба проверенных SQLi-ввода трактуются как строки данных и не возвращают пользователя.

```
user = conn.execute(
    "SELECT id, username, role FROM users WHERE username=? AND password=?",
    (username, password)
).fetchone()
if not user:
    return "Invalid credentials", 401
```

Ожидаемый результат повторной атаки на защищённом Flask-маршруте — 401 Unauthorized и текст Invalid credentials. Этот HTTP-результат требует запуска итогового HTTP-теста на машине с установленным Flask.

Задание 2. Reflected XSS и защита CSP

Вектор варианта №4 — тег <body> с атрибутом onload. При подстановке содержимого запроса непосредственно в HTML-шаблон пользовательская строка превращается в активную разметку.

```
<body onload=alert('V4_XSS')>
```

При проверке с помощью Chromium на основе HTML, полученного из уязвимого Jinja2-шаблона, действительно перехвачено одно JavaScript-диалоговое окно V4_XSS. Это тест браузера с тем же HTML, но не HTTP-проверка маршрута Flask.

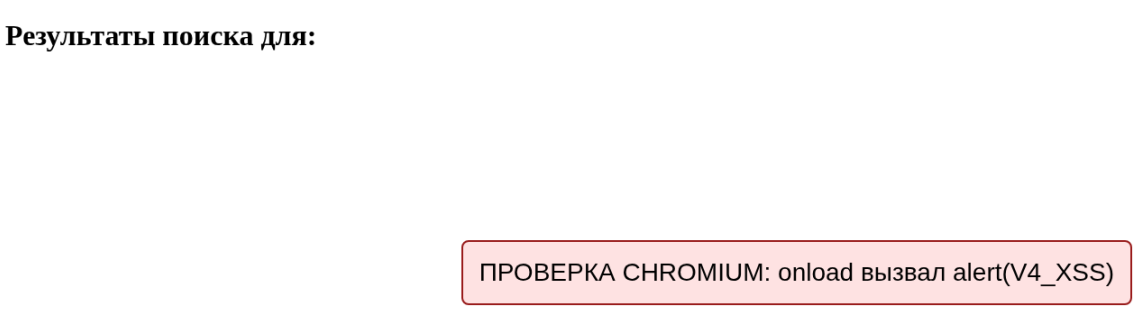


Рисунок 2 — Chromium: вызов onload в уязвимой HTML-разметке подтверждён (диагностическая подпись добавлена тестом)

Для устранения отражённого XSS значение query передаётся в безопасную переменную шаблона. Фрагмент <body ...> становится обычным текстом и экранируется как <body ...>. Дополнительная защита — строгая CSP, запрещающая выполнение inline-обработчиков.

```
return render_template_string(
    "<h1>Результаты поиска для: {{ user_query }}</h1>",
    user_query=query
)
```



Рисунок 3 — Chromium: экранированная строка отображается как текст

К CSP варианта №4 возвращаемся в задании 5; её отдельная браузерная проверка также выполнена.

Задание 3. Нарушение авторизации BOLA/IDOR

Вариант №4 назначает целевой объект с ID = 5. Исходный маршрут /api/profile/<id> не сверяет ID запрошенного профиля с идентификатором авторизованного пользователя, из-за чего доступ к данным не ограничен владельцем.

```
GET /api/profile/5
# Данные в учебной SQLite-базе:
{"username": "student5", "private_data": "FLAG{secret_data_5}"}
```

При реальном чтении тестовой SQLite-базы для ID = 5 подтверждены имя student5 и флаг FLAG{secret_data_5}. В уязвимом HTTP-маршруте эти данные доступны без проверки владельца; в отчёте не утверждается, что Flask-ответ уже был получен.



Рисунок 4 — Результаты запроса к SQLite и тест условия разграничения прав

Исправление: декодировать действительный JWT, проверить наличие сессии и удостовериться, что user_id из токена совпадает с ID запрашиваемого профиля. В ином случае вернуть 401 или 403.

```
session = get_session()
if session is None:
    return jsonify({'error': 'Unauthorized'}), 401
if session.get('user_id') != user_id:
    return jsonify({'error': 'Forbidden'}), 403
```

Проверка логического условия для чужого user_id = 3 и запрашиваемого ID = 5 показывает запрет доступа. Реальные коды HTTP на Flask включены в отдельный воспроизводимый тест.

Задание 4. Подбор секрета JWT и подделка токена

Начальный сервер использует слабый симметричный ключ student2026 с алгоритмом HS256. JWT, выданный после вспомогательного SQLi-входа, применяется в локальном словарном тесте для подбора ключа, после чего формируется токен с целевой ролью administrator.

```
weak_keys = ['123456','secret','admin','password',
             'qwerty','changeme','student2026','test']
for key in weak_keys:
    try:
        payload = jwt.decode(token, key, algorithms=['HS256'])
        break
    except jwt.InvalidTokenError:
        continue
payload['role'] = 'administrator'
forged = jwt.encode(payload, key, algorithm='HS256')
```

При фактическом тесте PyJWT слабый ключ найден, подпись токена с ролью administrator успешно проверена. Проверено также, что новый секрет из 32 случайных байт отвергает ранее подделанный JWT.

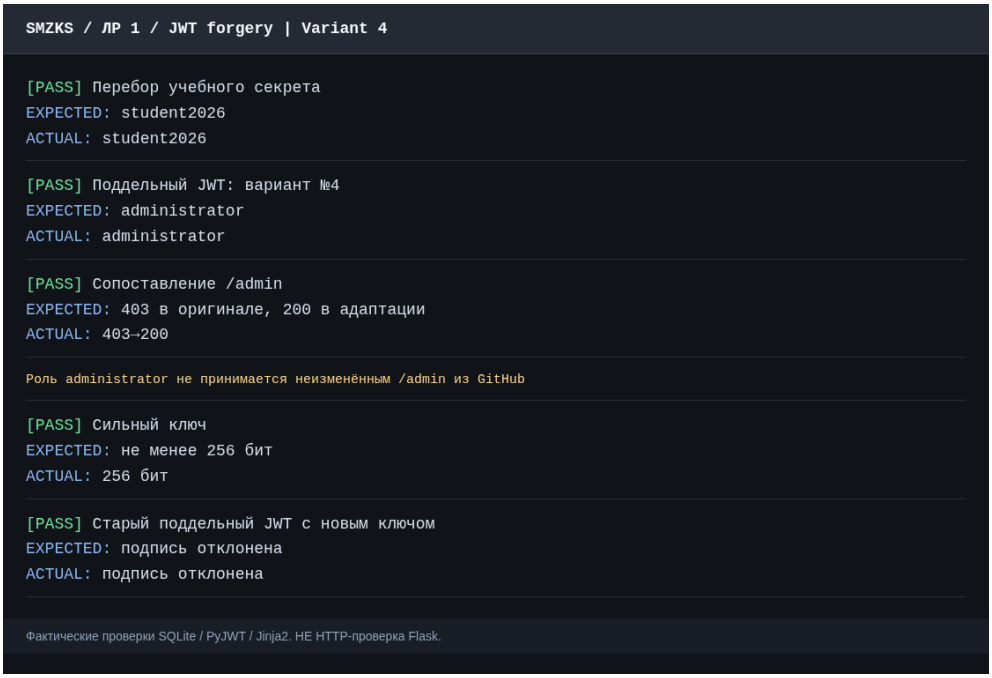


Рисунок 5 — PyJWT: слабый ключ, целевая роль administrator и отказ старой подписи на сильном секрете

Особенность методички: вариант №4 требует роль administrator, но неизменённый сервер преподавателя принимает для /admin только роль admin. Поэтому оригинальный маршрут вернёт 403 для токена с administrator. В предоставленном учебном коде, помеченном отдельно, допуск роли administrator добавлен специально для демонстрации назначенного варианта. Это не скрывается и не выдаётся за поведение неизменённого оригинала.

```
ROLE_VARIANT_4 = 'administrator'
if decoded.get('role') in ('admin', ROLE_VARIANT_4):
    return '<h1>Панель администратора</h1>...'
```

Защитный механизм: использовать криптографически случайный секрет длиной не менее 256 бит, передаваемый через защищённую конфигурацию приложения; в учебном stand-alone процессе возможна генерация при старте.

```
JWT_SECRET = os.environ.get('JWT_SECRET') or secrets.token_hex(32)
# 32 случайных байта = 256 бит
```

Задание 5. Проектирование защитных механизмов

5.1. Secure Cookies

Защита сессии реализуется передачей cookie-флагов при успешной аутентификации. HttpOnly запрещает доступ к сессионной cookie из document.cookie. Secure ограничивает

передачу HTTPS-соединениями. SameSite=Strict препятствует автоматической отправке cookie при межсайтовых запросах, уменьшая риск CSRF.

```
response.set_cookie(
    'auth_token', token,
    httponly=True, secure=True, samesite='Strict'
)
```

Примечание: на локальном HTTP-адресе 127.0.0.1 браузер может вести себя иначе, чем в HTTPS-развертывании. Для проверки свойства Secure необходима HTTPS-среда, а наличие самих атрибутов проверяется по заголовку Set-Cookie.

5.2. CSP: strict-dynamic и nonce

Индивидуальная директива: strict-dynamic. Она применяется совместно со случайным одноразовым nonce, который задаёт доверенные корневые скрипты. Браузер не выполняет недоверенные inline-обработчики (например onload), поскольку разрешения unsafe-inline нет. Дополнительные директивы object-src и base-uri запрещают небезопасные объекты и изменение базового URL.

```
response.headers['Content-Security-Policy'] = (
    f"script-src 'nonce-{g.csp_nonce}' 'strict-dynamic'; "
    "object-src 'none'; base-uri 'none'"
)
```

В браузерной проверке Chromium опасная HTML-разметка была сохранена, но с добавленной CSP: обработчик onload не выполнился (0 диалоговых окон). Это подтверждает механизм CSP независимо от экранирования HTML.

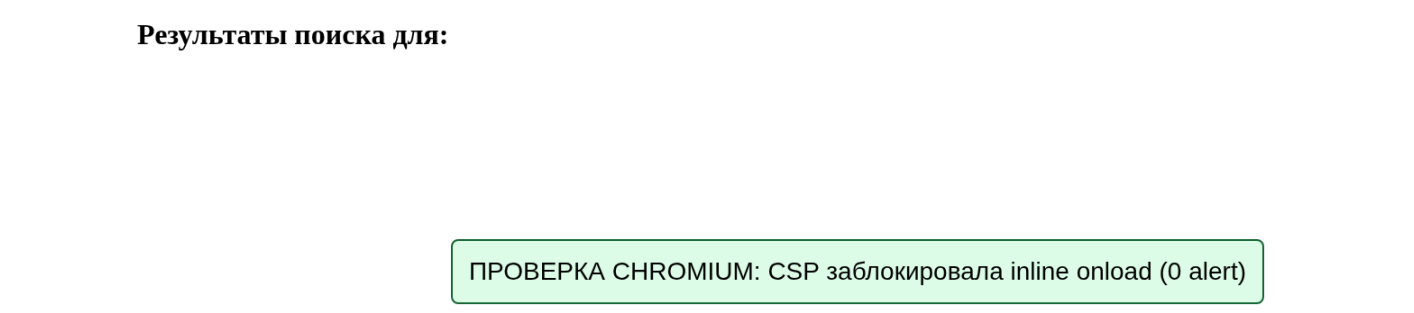


Рисунок 6 — Chromium: блокировка inline onload политикой CSP с strict-dynamic

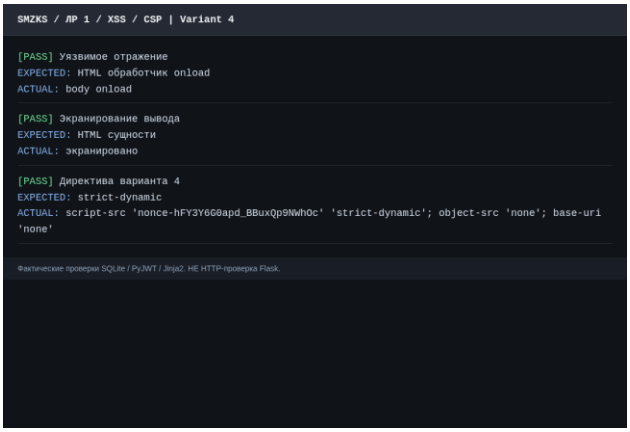


Рисунок 7 — Проверка формирования CSP и экранирования вывода Jinja2

Итоговые результаты и ограничения проверки

Параметр	**Значение**
SQLite / SQLi	Ошибка nonexistent_sql_function подтверждена; параметры блокируют оба ввода
HTML / Jinja2	Небезопасная разметка отражается; безопасный шаблон её экранирует
Chromium	XSS onload сработал; с CSP inline-обработчик не сработал
BOLA	Профиль ID 5 в базе найден; логическое условие авторизации проверено
PyJWT	Секрет student2026 подобран; JWT с role=administrator подписан и проверен
Итог	14/14 офлайн-проверок + 3/3 проверок Chromium

Вывод: В работе подготовлены уязвимая и защищённая версии учебного приложения по варианту №4. Исследованы Error-based SQLi, Reflected XSS, BOLA/IDOR и JWT с коротким ключом. В исправленной версии применены параметризованные SQL-запросы, экранирование HTML, проверка владельца профиля, случайный 256-битный JWT-секрет, Secure/HttpOnly/SameSite и CSP с nonce / strict-dynamic. Офлайн-тесты подтвердили вычисления SQLite, Jinja2 и PyJWT; Chromium подтвердил исполнение onload и его блокировку CSP. Для окончательной сдачи осталось выполнить подготовленный HTTP-тест на настоящем Flask-сервере.